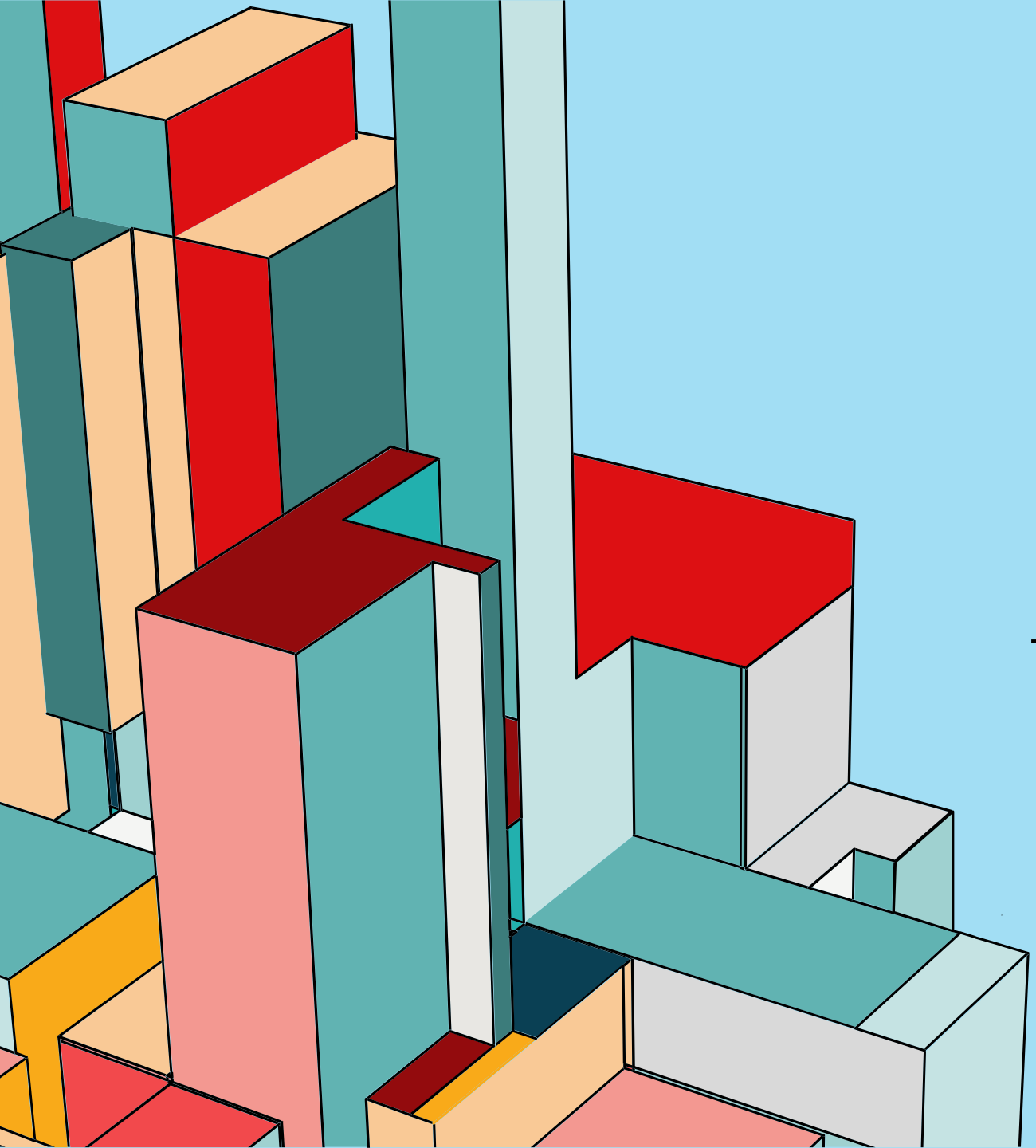


LECTURE 22
MORE MATCHING

CSIS 2300 Introduction to Algorithms, Spring 2026

Dr. Dan DiTursi

ANNOUNCEMENTS / REMINDERS

- Quantum Hackathon this weekend
- Exam 3 next Wednesday 4/15.
- A word about grade inquiries.

MATCHING REDUX (THE MATCH™)

Recall our discussion last time: Graduating medical students and openings in residency programs need to be matched up.

The actual problem is a bit more complicated than a simple yes/no: Every medical student ranks all of the programs they are interested in, and every program ranks all of the candidates they are interested in. Assignments are made using these rankings.

... How?

FIRST ATTEMPT: USING MAXIMUM BIPARTITE MATCHING

One obvious approach would be to use the preferences to generate a yes/no of some variety and then just run MBM.

- For example, we could simply say that any program that is ranked by a candidate at all is acceptable and vice versa.
  - But this approach ignores the ranking data – nothing here prevents everyone from getting their <u>last</u> choice.
- We could go with the opposite extreme: Only everyone's top choice (for both candidates and programs) is acceptable.
  - Almost certainly, most slots will go unfilled. It's even possible that we'll end up with ZERO edges in our graph!

FIRST ATTEMPT: USING MAXIMUM BIPARTITE MATCHING

We could try splitting the difference in some fashion: everyone's top-k or top-$p\%$ of choices are acceptable.

- But how do we find a good threshold?
- Furthermore, there are situations where <u>no</u> fixed threshold is going to produce universally good results: Some folks will still get a low-ranking option, while others remain unmatched.

An <u>iterative</u> approach might work: Let $k = 1$ then run MBM. Remove everyone that matches. Then let $k = 2$ and run MBM, then $k = 3$, and so on until everyone is matched...

- S L O W !

DEVELOPING AN INTUITION FOR THE PROBLEM

Let's take a step back to ask an important question: What does a good solution to this problem look like?

Consider a case with five programs A-E and five students V-Z.

Given these preferences, what would be a good matching?

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

Answer now in the Submitty poll.

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

A DESIRABLE PROPERTY

Given a proposed matching, what makes it good?

The default choice in the poll was: A-Z, B-Y, C-X, D-W, E-Z

Program C got its top choice. Everyone else is left wondering if they could have done better.

Say Program B happens to communicate with Student X. What happens?

B would rather hire X. X would rather work for B.

They'll switch! B will make an offer to X, and X will accept. This matching is <u>unstable</u>.

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

A DESIRABLE PROPERTY

Given a set of preferences, a matching is *stable* if and only if there is no pair P-S such that P would prefer S to its current match and S would prefer P to its current match. (i.e. No one can improve by unilaterally switching.)

In the medical residency context (and many others), stability is an important trait – The Match™ would not work if students/programs knew they could improve their lot by "shopping around" and cutting their own deals.

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

PROMULGATING STABILITY

Given a set of preferences, how do we find a <u>stable</u> matching? What algorithm might we use?

For that matter, are we even guaranteed that a stable matching exists at all? Perhaps there are preference lists that make stability impossible!

Fortunately, this turns out not to be the case – we will prove that <u>every</u> list of preferences has a stable matching.

What would be the best way of producing such a proof?

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

CONSTRUCTION!

The best way of demonstrating that something in math exists is by providing an algorithm that <u>produces</u> it.

Trying to solve the whole problem in one go seems really difficult, so let's try an iterative approach.

Specifically: Program A makes an offer to their top candidate, Student W.

> Having no other information besides her own preferences, should W accept this offer?

KEY CONCEPT: TENTATIVE ACCEPTANCE

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

On the one hand, W is really hoping for an offer from Program B.

On the other hand, W has no idea if such an offer is coming – it would be foolish to outright reject the offer from Program A.

The trick to this algorithm is that we will allow students to <u>tentatively</u> accept offers.

Let's continue: Program B is up next and <u>also</u> makes an offer to Student W.

W is happy to accept the offer from B…

… which means A is back in the market for a new hire.

KEY CONCEPT: TENTATIVE ACCEPTANCE

A: ~~W~~, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

Program C makes an offer to Student X; once again we have a tentative acceptance.

... and once again, that student immediately get a better offer, this time from Program D. So X tentatively accepts D's offer and sends C back into the market.

Then Program E makes an offer to Student W.

> W already has a better offer (from B) in hand, and thus it is safe for W to decline E's offer.

E tries again, offering Student Z, who is less than thrilled. But with no other offers in hand, Z will tentatively accept.

KEY CONCEPT: TENTATIVE ACCEPTANCE

A: ~~W~~, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: ~~X~~, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: ~~W~~, Z, V, Y, X	Z: B,C,A,D,E

We've come back around to A, whose next offer goes to Student Z.

This is an upgrade for Z, and so the offer is tentatively accepted, and E is on the hunt again.

B is still paired up and does nothing. Program C offers Student Y. With no other offers, Y tentatively accepts the offer from C.

D is also good. E's next choice is Student V, who tentatively accepts.

FINALIZING THE MATCH

A: ~~W~~, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: ~~X~~, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: ~~W~~, ~~Z~~, V, Y, X	Z: B,C,A,D,E

Observe that at this point, everyone has a tentative match.

That means we're done! No more offers will be made. Do you see why?

> Each program has <u>already</u> made an offer to (and been rejected by) every student that they like better than the one they are currently matched up with – any offer a program made at this point can only worsen its result.

Observe that this will always happen with this approach – it's a built-in property of the algorithm.

And this property also guarantees we have a <u>stable</u> matching!

PSEUDOCODE

Let's take a moment to write down our algorithm more precisely:

```
while any program is unfilled (and hasn't offered all
students yet):
  Pick any such program p
  Let s be the highest-ranked unoffered student in p's list
  p makes an offer to s
  If s is unattached:
    Attach p and s
  Else let p' be the current match for s:
    if s prefers p to p':
      Attach p and s; p' becomes unattached
Return the set of all attachments
```

A: ~~W~~, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: ~~X~~, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: ~~W~~, ~~Z~~, V, Y, X	Z: B,C,A,D,E

FORMALIZING THE PROOF

We will proceed by contradiction. Suppose this approach produced an unstable matching. Then there is some pair P-S that prefers each other to their actual matches. (Call these P-*s* and *p*-S.)

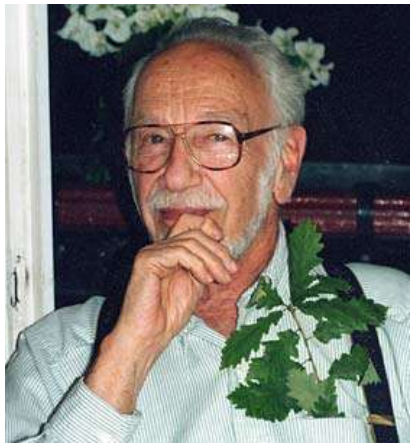
- How did P end up paired with *s*? P must have offered (and been rejected) by everyone they ranked higher than *s*, including S.
- But then how did S wind up with *p*? If they had seen an offer from P after tentatively accepting *p*'s offer, they would have switched up. And if they had seen P's offer first, they never would have rejected P in favor of *p*. Thus S must not have gotten an offer from P. **Contradiction!**

One final detail: The algorithm always terminates because there are at most n^2 offers to be made. (n positions $\times$ n candidates)

IF YOU WANT TO WIN A NOBEL PRIZE AS A COMPUTER SCIENTIST, BECOME AN ECONOMIST!

The algorithm we have been discussing is known as the *Gale-Shapley algorithm*, published by David Gale and Lloyd Shapley (left and center below) in 1962.

An economist named Alvin Roth (right below) recognized how applicable it was to <u>many</u> situations in economics and markets, and he published a number of enhancements.



In 2012, Shapley and Roth were jointly awarded the Nobel Prize in Economics. (Gale died in 2008.)

FURTHER EXPLORATIONS

Let's consider a smaller instance of the problem, with $n = 2$.

The preferences are:

A: Y,Z B: Z,Y Y: A,B Z: B,A

This one's easy! Everyone gets their first choice!

But let's change things up:

A: Y,Z B: Z,Y Y: B,A Z: A,B

This one's <u>easier</u>, because you can't get it wrong: Both possible matchings are stable!

FURTHER EXPLORATIONS

A: Y,Z B: Z,Y Y: B,A Z: A,B

Both possible matchings are stable… but they are definitely <u>not</u> equivalent.

One possibility (and the one produced by our algorithm) is A-Y, B-Z. Observe that in this matching, the programs A & B get exactly what they want. The students, however, very much don't.

What if we engage in a bit of role reversal? The students get to be the <u>proposers</u> (i.e. make the offers) and the programs get to be the <u>accepters</u>.

> Curious… now the students get exactly what <u>they</u> want and the programs get their last choices.

AN INHERENT ASYMMETRY

Our tiny example shows that there are situations where it matters a great deal whether one is on the proposing side or the accepting side of the algorithm.

This leads to a couple of questions:

- Is it always better to be the proposer?
- If so, is there any way to modify the algorithm to correct this asymmetry?

Let's go back to our original example for a moment.

RUNNING IT BACK

Execute the Gale-Shapley algorithm on these preferences, but treat the <u>students</u> (V-Z) as the proposers and programs (A-E) as the accepters.

Do we get the same matching as before, or a different one?

A: W, Z, V, Y, X	V: B,C,A,E,D
B: W, X, V, Z, Y	W: B,A,D,E,C
C: X, Y, Z, W, V	X: B,D,A,C,E
D: X, W, V, Z, Y	Y: A,B,C,D,E
E: W, Z, V, Y, X	Z: B,C,A,D,E

Answer now in the Submitty poll.

UNDERSPECIFIED

```
while any program is unfilled (and hasn’t offered all
students yet):
  Pick any such program p
  Let s be the highest-ranked unoffered student in p’s list
  p makes an offer to s
  If s is unattached:
    Attach p and s
  Else let p’ be the current match for s:
    if s prefers p to p’:
      Attach p and s; p’ becomes unattached
Return the set of all attachments
```

UNDERSPECIFIED

```
while any program is unfilled (and hasn't offered all
students yet):
  Pick any such program p
  Let s be the highest-ranked unoffered student in p's list
  p makes an offer to s
  If s is unattached:
    Attach p and s
  Else let p' be the current match for s:
    if s prefers p to p':
      Attach p and s; p' becomes unattached
Return the set of all attachments
```

How? Which one?

Does the order in which we select programs affect the outcome?

PROVING ASYMMETRY

It turns out that the question of sequencing the offers and the question of the proposer's advantage are closely tied together.

First of all, we know that there are cases where there is only <u>one</u> stable matching. (Our first tiny example, the Submitty example)

> In those cases, it clearly doesn't matter what order the offers are made; nor is there any asymmetry, since we'll get the same result either way.

Let's assume that the preference lists allow for multiple stable matchings. That means that a given program P_i could end up with one of several students; call these the *valid students* for P_i.

One of the valid students will be preferred over all the rest. Let best(P_i) represent the highest-ranked valid student of P_i.

PROVING ASYMMETRY

Let M^* be the matching that pairs P_i and best(P_i) for all i.

Objection!

PROVING ASYMMETRY

Let M^* be the matching that pairs P_i and best(P_i) for all i.

How do we even know that two (or more) different programs don't have the same best valid student?

We can let this slide, since it is an incidental result of this claim:

Claim: The Gale-Shapley algorithm <u>always</u> produces M^*, regardless of what order offers are made in!

Proof: By contradiction. Assume there is an execution of Gale-Shapley that does <u>not</u> match every P_i with best(P_i).

Perforce, at some point during this run, a program is rejected by a valid student. (Because offers are made in preference order.)

PROVING ASYMMETRY, CONT'D.

Consider the <u>first</u> instance in which a program is rejected by a valid student. Call this program P, and note that the student rejecting P must in fact be best(P). Call this student S.

P's rejection by S must be in favor of some other Program $\mathcal{P}$ that is preferred by S.

Now, since S is a valid student for P, there is <u>some</u> hypothetical stable matching M' in which P and S are paired up.

Q: In this stable matching M', who has $\mathcal{P}$ been paired up with?

PROVING ASYMMETRY, CONT'D.

Q: In stable matching M′, who has $\mathcal{P}$ been paired up with?

It can't be S (who is paired with P). So call this student s, and observe that both S and s are valid students for $\mathcal{P}$.

Recall that in our run of Gale-Shapley, the rejection of P by S was the <u>first</u> time any program had been rejected by a valid student. Thus, $\mathcal{P}$ cannot have been rejected by a valid student yet. And since offers are made in order of preference and $\mathcal{P}$ ends up with S in this run, it must be that $\mathcal{P}$ prefers S to s.

But now in M′ we have pairs P-S and $\mathcal{P}$–s. But $\mathcal{P}$ prefers S to s and we already had that S prefers $\mathcal{P}$ to P. Therefore, M′ is not stable. Contradiction!

AND IT GETS WORSE...

To recap: Every run of Gale-Shapley, regardless of the order in which offers are made, produces the specific stable matching M^* in which every program receives their top-ranked valid student.

Claim: M^* pairs each student with their <u>lowest</u>-ranked valid program!

Proof: Suppose there is some P-S in M^* where P is not the worst valid program for S. Then there is some <u>other</u> stable matching M′ that pairs S with another program p that S likes even less. So M′ contains the pairs P-s and p-S. Since S is P's best valid student, we have that P and S prefer each other to their current partners, and M′ is not stable. Contradiction!

FEEDBACK FORM

Please comment on today's class.